

PROYECTO INDIVIDUAL GYMCLI

Sistema de Gestión de un Gimnasio con Interfaz de Línea de Comandos

NOMBRE DEL ESTUDIANTE

Jakelin Mishel Quino Martínez

DOCENTES INVOLUCRADOS

Edwin Salas,
André López

CAMPUSLANDS

Salón Quetzal 1
Ruta NodeJS
Guatemala
2026

GESTIÓN DE UN GIMNASIO

1. Situación problema

Un gimnasio local gestiona actualmente sus operaciones con hojas de cálculo y registros en papel. Esta falta de una gestión óptima genera:

- Pérdida de seguimiento en el progreso físico cronológico de los clientes (peso, medidas, grasa corporal, evidencia fotográfica).
- Inconsistencias contables al registrar cobros o cancelaciones sin garantías de atomicidad e integridad en la base de datos.
- Ausencia de automatización en la generación de contratos y asignación de términos legales asociados a cada plan de entrenamiento.
- Inexistencia de validaciones de entrada en el ingreso de datos personales, lo que degrada la calidad de la información.

Para solventar esta problemática se plantea GymCLI, una aplicación CLI en Node.js con persistencia en MySQL, Programación Orientada a Objetos, principios SOLID y dos patrones de diseño (Repository y Factory). La herramienta automatiza la gestión de clientes, la generación de contratos (con firma simulada mediante un campo booleano firmado), el seguimiento físico y nutricional, y la contabilidad con transacciones SQL reales.

2. Levantamiento de requerimientos

El levantamiento se ejecutó analizando el flujo operativo de un gimnasio tipo. Se definieron 6 módulos principales:

- **Gestión de Clientes:** altas, bajas lógicas, actualizaciones, consultas y vinculación a planes.
- **Gestión de Planes de Entrenamiento:** definición de objetivos, duraciones, niveles y gestión de estados.
- **Contratos:** generación automática al asignar un plan, con términos, vigencia, firma simulada y estado.
- **Seguimiento Físico:** registro cronológico de métricas antropométricas con estados (activo/inactivo) y evidencia fotográfica.
- **Nutrición:** creación de planes alimenticios, diarios calóricos y reportes semanales.
- **Gestión Financiera:** control de ingresos/egresos y consultas de balances bajo transacciones SQL explícitas.

Nota: El sistema no implementa autenticación real de usuarios. Los roles “Administrador” y “Entrenador” mencionados en las historias de usuario se refieren al operador único de la CLI (el dueño del gimnasio o entrenador personal). Se documenta como simplificación de alcance, consistente con que el proyecto se entrega de forma individual.

3. Requerimientos

3.1 Requerimientos funcionales (RF)

Código	Descripción
RF-01	Registrar clientes con datos validados (nombre, correo con sintaxis válida, teléfono).
RF-02	Listar, actualizar y eliminar (borrado lógico) clientes.

Código	Descripción
RF-03	Asociar un cliente a uno o varios planes de entrenamiento.
RF-04	Crear planes de entrenamiento con nombre, duración (meses), metas físicas, nivel y precio.
RF-05	Generar automáticamente un contrato al asignar un plan a un cliente, con precio, condiciones, fechas, estado 'activo' y campo firmado.
RF-05a	Bloquear el registro de Seguimiento Físico o Nutrición mientras el contrato asociado tenga firmado = false.
RF-06	Renovar, finalizar o cancelar un plan de entrenamiento.
RF-07	Al cancelar un plan: si firmado = true, ejecutar transacción que marca contrato 'cancelado', plan_cliente 'cancelado', seguimiento 'inactivo' y nutrición 'inactivo' (si existen activos). Si firmado = false, marcar solo contrato y plan_cliente como 'cancelado' (sin cascada). ROLLBACK si falla cualquier sentencia.
RF-08	Registrar avances semanales de seguimiento físico (peso, % grasa, medidas por zona corporal, ruta de foto, notas) con estado 'activo' por defecto.
RF-09	Consultar el historial de seguimiento físico en orden cronológico.
RF-10	Eliminar/deshabilitar seguimiento físico: eliminación física solo si el plan asociado no está activo; si está activo, borrado lógico. Operación dentro de try/catch; ROLLBACK si falla la eliminación física.
RF-11	Crear planes de alimentación asociados al cliente y a su plan de entrenamiento (vía plan_cliente_id), con estados 'activo', 'inactivo', 'finalizado'.
RF-12	Registrar consumo diario de alimentos con estimación de calorías (ingreso manual de nombre y calorías).
RF-13	Generar reportes nutricionales semanales con acumulado y promedio calórico.
RF-14	Registrar ingresos y egresos.
RF-15	Ejecutar todas las operaciones financieras dentro de transacciones SQL reales (START TRANSACTION, COMMIT, ROLLBACK).
RF-16	Consultar balances financieros filtrados por rango de fechas o por cliente.
RF-17	Listar planes de entrenamiento disponibles y filtrar por nivel.
RF-18	Consultar el contrato activo de un cliente.
RF-19	Interfaz interactiva de menú en consola con navegación por flechas y confirmación explícita para acciones destructivas.
RF-20	Al finalizar un plan por vigencia: transacción que marca contrato 'finalizado', plan_cliente 'finalizado', nutrición 'finalizado' (si existe activa). Seguimiento permanece activo (histórico).
RF-21	Al renovar un plan: transacción que finaliza el contrato anterior, extiende fecha_fin del plan_cliente y crea un nuevo contrato 'activo' con firmado = false. plan_cliente permanece 'activo'.

3.2 Requerimientos no funcionales (RNF)

Código	Descripción
RNF-01	Lenguaje y paradigma: Node.js con Programación Orientada a Objetos.
RNF-02	Arquitectura: principios SOLID y dos patrones de diseño (Repository, Factory).
RNF-03	Persistencia: MySQL mediante mysql2/promise (se elige sobre el driver oficial mysql por su soporte nativo de Promises).
RNF-04	Validaciones de dominio en /models: clases con validación de tipos, rangos y formatos por campo.
RNF-05	Interfaz CLI con inquirer, chalk y dotenv.
RNF-06	Estructura de carpetas: /models, /services, /commands, /config, /utils, /repositories, /factories.
RNF-07	Transacciones SQL explícitas en toda operación multi-tabla o de alta criticidad (pagos, cancelación, finalización, renovación).
RNF-08	Entregables: README.md, .gitignore, .env.example, schema.sql (DDL + seed), Conventional Commits, video ≤ 7 min.
RNF-09	Pruebas unitarias con Node Test Runner sobre las validaciones críticas de los modelos.
RNF-10	Proyecto académico, sin licencia comercial.

4. Historias de usuario

4.1 Product Backlog priorizado

ID	Historia	Prioridad	Sprint
HU-01	Registrar y validar clientes	Alta	1
HU-02	Asignar plan y generar contrato automático	Alta	1
HU-03a	Cancelación de plan con rollback	Alta	2
HU-03b	Finalización de plan por vigencia	Alta	2
HU-03c	Renovación de plan (Modelo A)	Media	2
HU-04	Control financiero transaccional	Alta	2
HU-05	Seguimiento físico cronológico con evidencia fotográfica	Media	3
HU-06	Plan nutricional y reporte calórico	Media	3
HU-07	Interfaz interactiva CLI	Alta	1
HU-08	Consulta de planes y contrato activo	Media	1

4.2 Historias de usuario detalladas

HISTORIA DE USUARIO HU-01	
Prioridad / RF	Alta RF-01, RF-02, RNF-04
Actor	Operador
Nombre	Registrar y validar clientes
Descripción	Como operador quiero registrar clientes con datos válidos para mantener un expediente ordenado.
Criterios de aceptación	• Valida correo con formato usuario@dominio.com. • Valida teléfono (8-15 dígitos, + opcional). • Valida nombre entre 2 y 100 caracteres. • Rechaza correos duplicados con mensaje amigable (maneja ER_DUP_ENTRY). • Confirma el alta con colores en consola (chalk).
Restricciones	• Soporta al menos 1000 clientes sin degradación.

HISTORIA DE USUARIO HU-02	
Prioridad / RF	Alta RF-03, RF-04, RF-05, RNF-02
Actor	Operador
Nombre	Asignar plan y generar contrato automático
Descripción	Como operador quiero asignar un plan a un cliente generando el contrato automáticamente.
Funcionalidad	ContratoFactory.js crea el objeto Contrato con las fechas ya calculadas; ContratoRepository.js persiste dentro de una transacción gestionada por ContratoService.js.
Criterios de aceptación	• Selecciona cliente activo y plan activo del catálogo. • Valida que no exista ya un plan activo del mismo tipo para ese cliente. • Contrato se crea con firmado = false. • Solicita confirmación para firmar; si acepta, firmado = true en la misma transacción. • Fechas de plan_cliente y contrato son idénticas (calculadas una sola vez).
Restricciones	• No se puede asignar a cliente inexistente o dado de baja. • Solo un contrato activo por plan_cliente (validado en la capa de servicio). • Un contrato sin firmar no permite finalización ni renovación; solo firmarse o cancelarse sin cascada. • No se permite eliminación física de contratos; todo cambio de estado es lógico.

HISTORIA DE USUARIO HU-03a	
Prioridad / RF	Alta RF-06, RF-07, RF-05a, RNF-03
Actor	Operador
Nombre	Cancelación de plan con rollback
Descripción	Como operador quiero cancelar un plan revocando contrato, seguimiento y nutrición de forma atómica.
Criterios de aceptación	• Valida que plan_cliente.estado === 'activo' antes de cancelar. • Si contrato.firmado = true: contrato 'cancelado', plan_cliente 'cancelado', seguimiento 'inactivo', nutrición 'inactivo' (si existen activos). • Si contrato.firmado = false: solo contrato y plan_cliente 'cancelado' (RF-05a garantiza que no puede haber seguimiento/nutrición activos en este caso). • ROLLBACK si falla cualquier sentencia. • Pregunta si registrar egreso por devolución; si acepta, en la misma transacción.
Restricciones	• Solo se cancela un contrato en estado 'activo'.

HISTORIA DE USUARIO HU-03b	
Prioridad / RF	Alta RF-06, RF-20
Actor	Operador
Nombre	Finalización de plan por vigencia
Descripción	Como operador quiero finalizar un plan cuando vence por cumplimiento de vigencia.
Criterios de aceptación	• Requiere contrato.firmado = true. • contrato 'finalizado', plan_cliente 'finalizado', nutrición 'finalizado' (si existe activa). • Seguimiento permanece activo (histórico). • ROLLBACK si falla.
Restricciones	• Solo se finaliza un contrato en estado 'activo' y con firmado = true.

HISTORIA DE USUARIO HU-03c	
Prioridad / RF	Media RF-06, RF-21
Actor	Operador
Nombre	Renovación de plan (Modelo A - extensión)
Descripción	Como operador quiero renovar un plan activo o finalizado extendiendo su vigencia.
Criterios de aceptación	• Requiere contrato.firmado = true. • Marca contrato anterior 'finalizado'. • Actualiza fecha_fin del plan_cliente sumando la nueva duración. • Crea nuevo contrato 'activo' con firmado = false. • plan_cliente permanece 'activo'; seguimiento y nutrición no se reinician. • ROLLBACK si falla.
Restricciones	• Solo se renueva un contrato en estado 'activo' o 'finalizado' y con firmado = true. • El estado 'renovado' no existe en el enum: la renovación se refleja en el histórico de contratos.

HISTORIA DE USUARIO HU-04	
Prioridad / RF	Alta RF-14, RF-15, RF-16, RNF-03
Actor	Operador
Nombre	Control financiero transaccional
Descripción	Como operador quiero registrar ingresos y egresos en transacciones reales para garantizar un balance consistente.
Criterios de aceptación	• Registra ingresos y egresos con tipo y monto positivo; el signo contable se deriva del tipo. • ROLLBACK si la inserción falla. • Balance muestra total ingresos, total egresos y neto. • Filtra balances por rango de fechas o por cliente. • Si no hay transacciones en el rango, el balance muestra Q0.00 (no error).
Restricciones	• Monto estrictamente > 0. Tipo enum validado. cliente_id es nullable (egresos operativos no requieren cliente).

HISTORIA DE USUARIO HU-05	
Prioridad / RF	Media RF-08, RF-09, RF-10
Actor	Entrenador
Nombre	Seguimiento físico cronológico con evidencia fotográfica
Descripción	Como entrenador quiero registrar el avance corporal semanal (incluyendo evidencia fotográfica) y visualizar su evolución cronológica.
Criterios de aceptación	• Registra avances validando rangos de peso, grasa y cada medida por zona corporal. • Registra una ruta de foto (string, máx. 255 caracteres) relativa a /evidencias/fotos/; la imagen no se guarda en la BD. • Despliega historial cronológico. • Eliminación física solo si el plan asociado no está activo, dentro de try/catch. Si el plan está activo, borrado lógico (estado = 'inactivo'). • Si se intenta eliminar físicamente un registro de un plan activo, el sistema rechaza la operación y ofrece borrado lógico.
Restricciones	• Peso 20-300 kg. % Grasa 3-60. Medidas por zona: positivas, máx. 300 cm. Notas máx. 500 caracteres.

HISTORIA DE USUARIO HU-06	
Prioridad / RF	Media RF-11, RF-12, RF-13
Actor	Entrenador
Nombre	Plan nutricional y reporte calórico
Descripción	Como entrenador quiero asignar planes de alimentación y registrar comidas para obtener un reporte semanal.
Criterios de aceptación	• Vincula el plan nutricional a plan_cliente_id (ya asocia cliente y plan a la vez) con estado 'activo'. • Registra alimentos diarios por nombre y calorías (ingreso manual). • Genera reporte semanal con acumulado y promedio calórico diario.
Restricciones	• Un cliente no puede tener dos planes nutricionales activos simultáneamente. • Si plan_cliente se cancela → nutrición 'inactivo' en la misma transacción. • Si plan_cliente se finaliza → nutrición 'finalizado' en la misma transacción.

HISTORIA DE USUARIO HU-07	
Prioridad / RF	Alta RF-19, RNF-05, RNF-06, RNF-02
Actor	Operador
Nombre	Interfaz interactiva CLI
Descripción	Como operador quiero un menú interactivo para navegar entre las funciones de la aplicación.
Criterios de aceptación	• Menú con opciones: Clientes, Planes, Contratos, Finanzas, Seguimiento, Nutrición, Salir. • Navegación con flechas (inquirir). • Captura errores sin cerrar la app (try/catch por módulo + process.on('unhandledRejection')). • Acciones destructivas solicitan confirmación previa.
Restricciones	• El bucle principal debe manejar errores no controlados sin cerrar el proceso.

HISTORIA DE USUARIO HU-08	
Prioridad / RF	Media RF-17, RF-18
Actor	Operador
Nombre	Consulta de planes y contrato activo
Descripción	Como operador quiero consultar los planes disponibles y el contrato activo de un cliente.
Criterios de aceptación	• Lista planes activos del catálogo y filtra por nivel. • Consulta contrato activo por cliente (ID o email). • Muestra plan, fechas, precio y estado de firma.
Restricciones	• Si no hay contrato activo, lo indica claramente.

5. Metodología

5.1 Marco de trabajo y roles

El proyecto se desarrolla bajo SCRUM con adaptación para desarrollo individual (Personal Agile Framework). Al ser un proyecto de 1 integrante, los tres roles son asumidos por la misma persona en distintos momentos del día, documentado como simplificación de alcance. La plataforma a usar será 'Moday Dev'.

- **Product Owner:** Jakelin Quino - prioriza el Product Backlog, define criterios de aceptación y valida entregables.
- **Scrum Master:** Jakelin Quino - facilita las ceremonias y elimina impedimentos.
- **Developer:** Jakelin Quino— implementación técnica, pruebas y documentación.

5.2 Ceremonias adaptadas

- Sprint Planning diario (15 min): al inicio de cada día se define el objetivo de la iteración.
- Daily Stand-Up: no aplica (ciclo diario); se sustituye por bitácora de progreso.
- Sprint Review diario (15 min): al final del día se verifica lo completado.
- Sprint Retrospective: solo al final del proyecto (día 5), con lecciones aprendidas.

5.3 Cronograma

Día	Fecha	Sprint	Alcance
1	24 sep	Sprint 1	Setup + BD + modelos + CLI base + clientes + planes + contratos
2	25 sep	Sprint 2	Finanzas + cancelación / finalización / renovación con rollback
3	26 sep	Sprint 3	Seguimiento + nutrición + tests
4	27 sep	Sprint 4	Refactor + README + grabar video
5	28 sep	Sprint5	Editar video + terminar PDF

6. Evidencia de plataforma de trabajo

6.1 Repositorio Git

- Link: https://github.com/JakelinMishelQuinoMartinez/gymcli_nodejs

6.3 Tablero Monday Dev

- Link: <https://jakelinquino303s-team.monday.com/boards/18432730047>
- Capturas de forma resumida del avance y organización de los tableros.

> 🌟 Sprint 1: Setup, Modelos & Gestión Pr	Estado	Prioridad	Sprint 0 Total
4 Elementos / 18 Subelementos			
> 📄 Sprint 2: Transacciones Financieras	Estado	Prioridad	Sprint 0 Total
4 Elementos / 17 Subelementos			
> 🏃 Sprint 3: Seguimiento Físico, Nutrició	Estado	Prioridad	Sprint 0 Total
3 Elementos / 10 Subelementos			
> 🛠 Sprint 4: Refactor, Documentación &	Estado	Prioridad	Sprint 0 Total
3 Elementos / 7 Subelementos			
> 📁 Sprint 5: Cierre, Edición & Entrega PD	Estado	Prioridad	Sprint 0 Total
1 Elemento / 3 Subelementos			
> 🚀 Backlog	Estado	Prioridad	Sprint 18 Total
10 Elementos / 43 Subelementos			

>  Sprint 1: Setup, Modelos & Gestión Pr 4 Elementos / 18 Subelementos				Estado 	Prioridad 		Sprint 0 Total
>  Sprint 2: Transacciones Financieras 4 Elementos / 17 Subelementos				Estado 	Prioridad 		Sprint 0 Total
>  Sprint 3: Seguimiento Físico, Nutrició 3 Elementos / 10 Subelementos				Estado 	Prioridad 		Sprint 0 Total
>  Sprint 4: Refactor, Documentación & 3 Elementos / 7 Subelementos				Estado 	Prioridad 		Sprint 0 Total
>  Sprint 5: Cierre, Edición & Entrega PD 1 Elemento / 3 Subelementos				Estado 	Prioridad 		Sprint 0 Total
>  Backlog 10 Elementos / 43 Subelementos				Estado 	Prioridad 		Sprint 18 Total

>  Sprint 1: Setup, Modelos & Gestión Pr 4 Elementos / 18 Subelementos				Estado 	Prioridad 		Sprint 0 Total
>  Sprint 2: Transacciones Financieras 4 Elementos / 17 Subelementos				Estado 	Prioridad 		Sprint 0 Total
>  Sprint 3: Seguimiento Físico, Nutrició 3 Elementos / 10 Subelementos				Estado 	Prioridad 		Sprint 0 Total
>  Sprint 4: Refactor, Documentación & 3 Elementos / 7 Subelementos				Estado 	Prioridad 		Sprint 0 Total
>  Sprint 5: Cierre, Edición & Entrega PD 1 Elemento / 3 Subelementos				Estado 	Prioridad 		Sprint 0 Total
>  Backlog 10 Elementos / 43 Subelementos				Estado 	Prioridad 		Sprint 18 Total

>  Sprint 1: Setup, Modelos & Gestión Pr 4 Elementos / 18 Subelementos				Estado 	Prioridad 		Sprint 0 Total
>  Sprint 2: Transacciones Financieras 4 Elementos / 17 Subelementos				Estado 	Prioridad 		Sprint 0 Total
>  Sprint 3: Seguimiento Físico, Nutrició 3 Elementos / 10 Subelementos				Estado 	Prioridad 		Sprint 0 Total
>  Sprint 4: Refactor, Documentación & 3 Elementos / 7 Subelementos				Estado 	Prioridad 		Sprint 0 Total
>  Sprint 5: Cierre, Edición & Entrega PD 1 Elemento / 3 Subelementos				Estado 	Prioridad 		Sprint 0 Total
>  Backlog 10 Elementos / 43 Subelementos				Estado 	Prioridad 		Sprint 18 Total

>  Sprint 1: Setup, Modelos & Gestión Pr 4 Elementos / 18 Subelementos				Estado 	Prioridad 		Sprint 0 Total
>  Sprint 2: Transacciones Financieras 4 Elementos / 17 Subelementos				Estado 	Prioridad 		Sprint 0 Total
>  Sprint 3: Seguimiento Físico, Nutrició 3 Elementos / 10 Subelementos				Estado 	Prioridad 		Sprint 0 Total
>  Sprint 4: Refactor, Documentación & 3 Elementos / 7 Subelementos				Estado 	Prioridad 		Sprint 0 Total
>  Sprint 5: Cierre, Edición & Entrega PD 1 Elemento / 3 Subelementos				Estado 	Prioridad 		Sprint 0 Total
>  Backlog 10 Elementos / 43 Subelementos				Estado 	Prioridad 		Sprint 18 Total

7. Conclusiones

7.1 Conclusiones generales

- Se logró resolver el problema del gimnasio reemplazando métodos de registros físicos por una aplicación en consola rápida y organizada.
- Información segura y sin errores: Gracias a las validaciones y a las transacciones de MySQL, los datos de los clientes, pagos y contratos ahora están protegidos. Si algo falla a mitad de un proceso, el sistema cancela los cambios para evitar datos incompletos.
- Control total del cliente: El sistema permite llevar un registro completo del progreso físico, las dietas y el dinero del gimnasio de forma clara, ordenada y sin complicaciones.

7.2 Conclusiones de la retrospectiva

- Organización: Trabajar con el tablero de Monday Dev y estructurar el proyecto por sprints me ayudó a llevar un orden con los requerimientos que el proyecto describe, aunque se me dificultó controlar el tiempo de desarrollo, se logró completar el proyecto con las funcionalidades básicas.
- Aprender a resolver fallos: Durante la marcha surgieron dudas con algunas consultas de la base de datos y código de JavaScript pero se solucionaron investigando y revisando notas.
- Crecimiento personal y técnico: Desarrollar este proyecto de forma individual me permitió poner en práctica todo lo aprendido en la ruta de Node.js y comprender el flujo de trabajo que se maneja.